INFORMATICA · CLASSE QUINTA

Progettazione logica, normalizzazione e SQL

Dal modello ER alle tabelle, associazioni particolari, integrità referenziale, forme normali, comandi e funzioni di aggregazione SQL

Come leggere questi appunti. I riquadri blu contengono le **definizioni** da sapere, quelli verdi gli **esempi**, quelli gialli i punti in cui è **facile sbagliare**. Negli schemi logici la chiave primaria è **sottolineata** e le chiavi esterne sono in *corsivo* con (FK). In fondo trovi uno schema riassuntivo, le domande di autoverifica con le risposte e l'elenco delle correzioni rispetto agli appunti.

1. La progettazione logica

Progettare un database avviene per gradi. Prima si disegna lo **schema ER** (Entità–Relazioni), che descrive la realtà da rappresentare: quali entità ci sono, che attributi hanno, come sono collegate. Lo schema ER però è un disegno concettuale: un DBMS non sa cosa farsene di rettangoli e rombi. Serve quindi un passo intermedio che trasformi il disegno in qualcosa di realizzabile.

> **DEFINIZIONE**
> La **progettazione logica** trasforma lo schema ER in uno **schema logico**, cioè l'elenco delle tabelle con i loro attributi e le loro chiavi. A differenza dell'ER, lo schema logico è **pronto per essere implementato** in un DBMS.

Lo schema logico si scrive di solito in forma compatta: nome della tabella e, tra parentesi, i suoi attributi.

```
CLIENTI(id, nome)
```

1.1 I passaggi dall'ER allo schema logico

Alcuni costrutti dell'ER non hanno un equivalente diretto nelle tabelle. Prima della traduzione bisogna quindi «ripulire» lo schema con questi passaggi:

1. **Eliminare eventuali ridondanze**: informazioni ripetute o ricavabili da altre si tolgono, perché in una tabella creerebbero dati doppi da tenere allineati.
2. **Eliminare le generalizzazioni** (le gerarchie padre–figlio tra entità) **senza perdere informazioni**: le tabelle non conoscono l'ereditarietà, quindi la gerarchia va trasformata in tabelle normali.
3. **Partizionare o accorpare** entità e/o associazioni: un'entità si può dividere in più tabelle, oppure due elementi si possono unire in una sola tabella, se questo rende lo schema più semplice da usare.

Negli appunti il primo punto è poco leggibile dopo «eliminazione di eventuali»: si legge con certezza solo «ridondanze».

1.2 Tradurre le associazioni in chiavi esterne

Le associazioni dell'ER diventano **chiavi esterne** (FK, *foreign key*): un attributo di una tabella che contiene il valore della chiave primaria di un'altra tabella, e così le collega. Dove mettere la FK dipende dalle **cardinalità**.

Nel caso più comune, **(1,1) da una parte e (1,N) dall'altra**, la FK va nella tabella del lato (1,1): ogni riga di quella tabella è legata a una e una sola riga dell'altra, quindi basta una colonna.

Con associazioni come **(0,1)–(1,N)**, invece, il lato (0,1) può anche non essere collegato a nulla. Ci sono due soluzioni:

- mettere la FK nella tabella del lato (0,1) e renderla **nullable** (può valere NULL quando il collegamento non c'è);
- creare una **tabella dedicata** all'associazione, con le due FK, che contiene una riga solo per i collegamenti che esistono davvero.

> **ATTENZIONE**
> Lo zero nella cardinalità minima significa «partecipazione facoltativa». Se si mette la FK senza permettere il NULL, si obbliga ogni riga ad avere un collegamento che nella realtà potrebbe non esserci.

2. Casi particolari di associazioni

2.1 Associazione ricorsiva

Un'**associazione ricorsiva** collega un'entità con se stessa. Nell'esempio, un ministro **succede** a un altro ministro: entrambi i lati dell'associazione sono la stessa entità MINISTRO, ma con due ruoli diversi (precedente e successore).

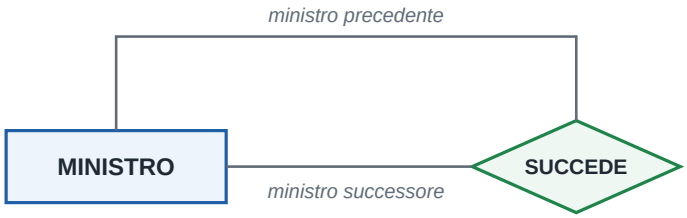


Figura 1 – Associazione ricorsiva: un ministro succede a un altro ministro

Nella tabella dell'associazione ci sono quindi **due FK che puntano alla stessa tabella** MINISTRO, una per ruolo. Insieme formano la chiave primaria della tabella SUCCESSIONE.

```
MINISTRO(id, nome, …)
SUCCESSIONE(ministro_prec (FK), ministro_succ (FK))
```

2.2 Associazione parallela

Si ha un'**associazione parallela** quando tra le stesse due entità esistono più associazioni con significati diversi. Un impiegato **lavora** in una o più città e **risiede** in una sola città: le entità sono le stesse, ma i legami sono due e vanno tradotti separatamente.

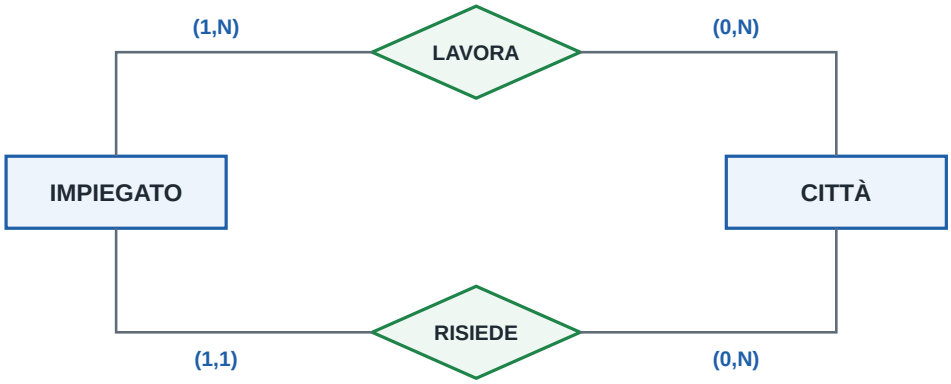


Figura 2 – Associazione parallela tra IMPIEGATO e CITTÀ

Ognuna segue la propria regola di traduzione:

- **RISIEDE** è (1,1)–(0,N): basta una FK `citta_residenza` dentro IMPIEGATO;
- **LAVORA** è (1,N)–(0,N), cioè molti a molti: serve una tabella apposita con le due FK.

```
IMPIEGATO(id, …, citta_residenza (FK))
CITTA(id, …)
LAVORA(impiegato (FK), citta (FK))
```

2.3 Identificatore esterno

A volte un'entità non si identifica da sola con i suoi attributi. Il numero di **matricola** di uno studente è unico solo dentro la sua università: due università diverse possono dare la stessa matricola a due studenti diversi. Per identificare lo studente serve quindi anche l'università.

> **DEFINIZIONE**
> Un **identificatore esterno** è un identificatore formato da attributi dell'entità più la chiave di un'altra entità a cui è collegata.

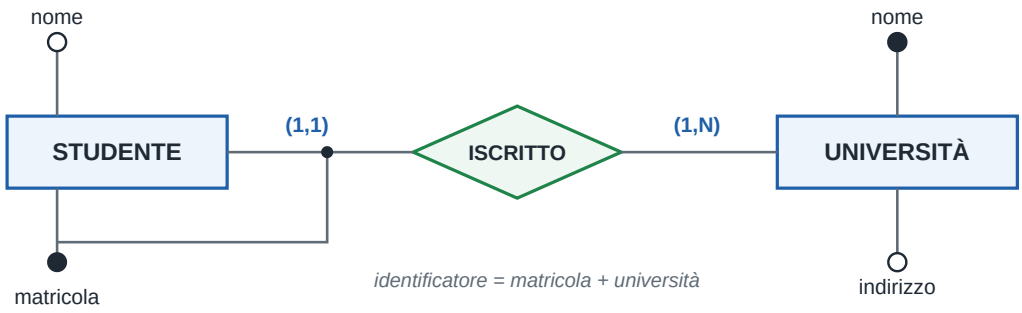


Figura 3 – Lo studente è identificato da matricola più università

Nello schema logico la chiave primaria di STUDENTE è composta da due attributi, e uno dei due è anche chiave esterna:

```
STUDENTE(matricola, universita (FK), nome)
```

> **ATTENZIONE**
> L'identificatore esterno si può usare **solo con cardinalità (1,1) → (1,N)**: l'entità identificata (STUDENTE) deve essere collegata a una e una sola università, altrimenti non si saprebbe quale usare per identificarla.

2.4 Vincolo di integrità referenziale

> **DEFINIZIONE**
> Il **vincolo di integrità referenziale** stabilisce che una FK non può assumere un valore qualsiasi: ogni suo valore deve **corrispondere a una PK esistente** nella tabella a cui punta.

Serve a mantenere la **coerenza delle associazioni** con la tabella esterna. Se un impiegato avesse `citta_residenza = 57` ma nella tabella CITTA non esistesse nessuna città con id 57, il collegamento punterebbe nel vuoto. Il vincolo impedisce al DBMS di accettare dati del genere.

3. Esercizi svolti: dall'ER allo schema logico

3.1 Azienda: reparti, dipendenti, uffici e task

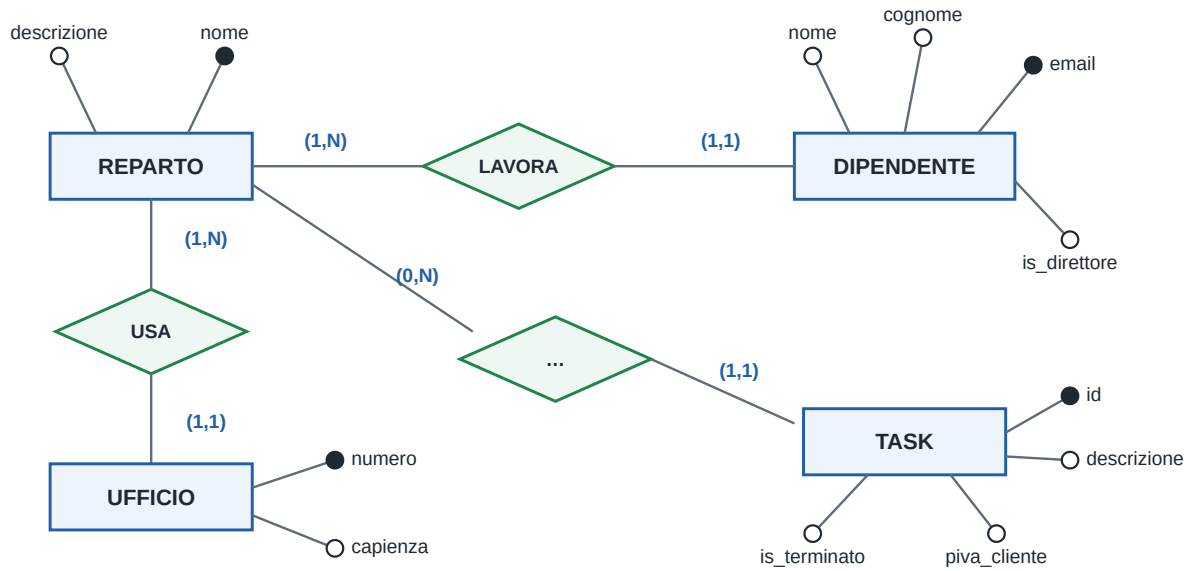


Figura 4 – Schema ER dell'azienda (pallino pieno = identificatore). Negli appunti il nome dell'associazione tra REPARTO e TASK non si legge.

Le tre associazioni partono tutte da REPARTO e hanno dall'altro lato cardinalità (1,1): ogni dipendente lavora in un solo reparto, ogni ufficio è usato da un solo reparto, ogni task appartiene a un solo reparto. In tutti e tre i casi la traduzione è la stessa: una FK verso REPARTO nella tabella del lato (1,1). Non servono tabelle aggiuntive.

```
REPARTO(nome, descrizione)
DIPENDENTE(email, nome, cognome, is_direttore, reparto (FK))
UFFICIO(numero, capienza, reparto (FK))
TASK(id, descrizione, is_terminato, piva_cliente, reparto (FK))
```

ESEMPIO · PERCHÉ LA FK VA SUL LATO (1,1)

Se mettessimo la FK in REPARTO, un reparto con dieci dipendenti dovrebbe contenere dieci email in una sola colonna: non si può. In DIPENDENTE, invece, ogni riga ha un solo reparto, quindi una colonna basta.

3.2 Servizio taxi: corse, clienti e autisti

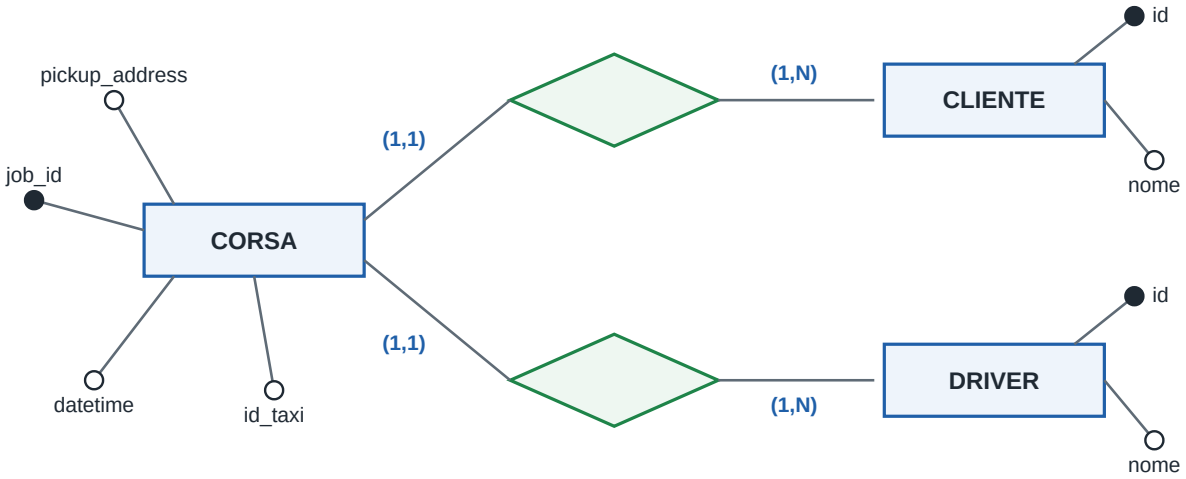


Figura 5 – Schema ER del servizio taxi

Ogni corsa ha un solo cliente e un solo autista (1,1), mentre un cliente e un autista fanno più corse (1,N). La tabella CORSE riceve quindi due FK, una verso CLIENTI e una verso AUTISTI.

```
CLIENTI(id, nome)
AUTISTI(id, nome)
CORSE(job_id, cliente_id (FK), autista_id (FK), id_taxi, datetime_pickup,
pickup_address)
```

Negli appunti la riga di CORSE è tagliata dal bordo del foglio dopo «pickup»: l'ultimo attributo è ricostruito dallo schema ER.

4. La normalizzazione

Uno schema logico può essere scritto correttamente e contenere comunque dei difetti: dati ripetuti, informazioni che si contraddicono, aggiornamenti da fare in più punti. La **normalizzazione** serve ad assicurarsi che lo schema logico sia corretto secondo i criteri della 1ª, 2ª e 3ª **forma normale** (NF, *normal form*).

4.1 Dipendenza funzionale

Per capire le forme normali serve prima un concetto di base.

> **DEFINIZIONE**
> Un attributo X è in **dipendenza funzionale** da un attributo Y se a ogni valore di Y è sempre associato lo stesso valore di X. Si scrive Y → X («Y determina X»).

> **ESEMPIO · CAP E CITTÀ**
> In una tabella anagrafica con sia il CAP sia la città, per un dato CAP la città è sempre la stessa: CAP → città. Se cento persone hanno lo stesso CAP, la stessa città viene scritta cento volte.

Il problema non è la dipendenza in sé, ma il fatto che CAP non sia la chiave della tabella: è proprio questa situazione che le forme normali eliminano.

4.2 Le tre forme normali

1NF. Una tabella è in prima forma normale se ha una **chiave primaria** e se tutti gli attributi sono **atomici**: niente liste di valori in una cella e niente sotto-attributi.

2NF. Ogni attributo non chiave deve dipendere da **tutta la PK**, non da una sua parte. Il problema si pone solo quando la PK è composta da più attributi: se un attributo dipende solo da uno di essi si ha una **dipendenza parziale**.

3NF. Non ci devono essere **dipendenze transitive**.

> **DEFINIZIONE**
> Una **dipendenza transitiva** si ha quando un attributo dipende da un altro attributo non chiave, che a sua volta dipende dalla chiave. Esempio: id → CAP → città.

Forma	Che cosa richiede	Che cosa elimina
1NF	Chiave primaria, attributi atomici	Liste e sotto-attributi
2NF	1NF + ogni attributo dipende da tutta la PK	Dipendenze parziali
3NF	2NF + nessun attributo dipende da un non-chiave	Dipendenze transitive

> **ATTENZIONE**
> Le forme normali sono una scala: ognuna richiede che sia rispettata anche quella precedente. Una tabella non può essere in 3NF se non è già in 2NF.

4.3 Come si normalizza: la decomposizione

Per portare una tabella in 2NF (e allo stesso modo in 3NF) si usa la **decomposizione**: si crea una **tabella per ogni attributo determinante**, cioè per ogni attributo da cui dipendono altri. Gli attributi dipendenti si spostano nella nuova tabella, e nella tabella di partenza resta solo il determinante, che diventa una FK.

> **ESEMPIO · DECOMPORRE L'ANAGRAFICA**
> ANAGRAFICA(**id**, nome, cap, città) diventa ANAGRAFICA(**id**, nome, *cap* (FK)) e CAP(**cap**, città). Ora la città è scritta una volta sola per ogni CAP.

5. SQL

5.1 Comandi sulle tabelle

Comando	Che cosa fa
`DROP TABLE`	Cancella la tabella intera: struttura e dati
`TRUNCATE TABLE`	Cancella tutte le tuple (righe) ma lascia la tabella
`ALTER TABLE`	Modifica la tabella: aggiunge o toglie colonne e vincoli

> **ATTENZIONE**
> DROP e TRUNCATE cancellano entrambi i dati, ma dopo un TRUNCATE la tabella esiste ancora (vuota) e si può riempire subito; dopo un DROP va ricreata da zero.

5.2 Controllare l'output

- `LIMIT n` mostra in output solo le prime **n tuple**.
- `SELECT DISTINCT` mostra il risultato **senza duplicati**.

5.3 UNION e JOIN

Entrambe combinano i dati di più tabelle, ma in direzioni diverse. La **JOIN** unisce le tabelle **di lato**: affianca le colonne di una alle colonne dell'altra. La **UNION** le unisce **in verticale**: mette le righe di una query sotto le righe dell'altra.

> **ATTENZIONE**
> Proprio perché impila le righe, la UNION vuole che le due query abbiano lo **stesso numero di colonne** e lo **stesso tipo di attributi**, colonna per colonna.

5.4 Funzioni di aggregazione

Aggregare vuol dire passare da tanti valori a uno solo: una funzione di aggregazione prende una colonna intera e restituisce un unico numero. Si scrivono dopo la SELECT.

Funzione	Risultato
`COUNT(*)`	Numero di righe, comprese quelle con valori NULL
`COUNT(campo)`	Numero di righe in cui *campo* non è NULL
`MIN`, `MAX`	Il valore minimo o massimo
`SUM`, `AVG`	La somma e la media

5.5 GROUP BY

Da sola, una funzione di aggregazione riassume tutta la tabella in un numero. Con **GROUP BY** si ottiene invece un risultato per ogni gruppo: la tabella viene divisa in sotto-tabelle, una per ogni valore

diverso della colonna indicata, e la funzione viene calcolata su ciascuna.

```
SELECT id_docente, COUNT(*)
FROM insegnamento
GROUP BY id_docente;
```

ESEMPIO · INSEGNAMENTI PER DOCENTE

GROUP BY crea una sotto-tabella per ogni valore diverso di `id_docente`; COUNT(*) conta le righe di ciascuna. Il risultato ha una riga per docente con il numero dei suoi insegnamenti.

6. Ripasso

6.1 In sintesi

1. La progettazione logica trasforma lo schema ER in tabelle pronte per essere implementate.
2. Prima della traduzione si eliminano ridondanze e generalizzazioni e si partizionano o accorpano entità e associazioni.
3. Con (1,1)–(1,N) la FK va nella tabella del lato (1,1).
4. Con (0,1)–(1,N) la FK è nullable, oppure si crea una tabella dedicata.
5. Associazione ricorsiva: due FK che puntano alla stessa tabella.
6. Associazione parallela: più associazioni tra le stesse entità, tradotte ognuna per conto suo.
7. Identificatore esterno: la PK include la chiave di un'altra entità; si usa solo con (1,1) → (1,N).
8. Integrità referenziale: ogni valore di una FK deve esistere come PK nella tabella a cui punta.
9. 1NF: PK e attributi atomici; 2NF: niente dipendenze parziali; 3NF: niente dipendenze transitive.
10. Si normalizza per decomposizione: una tabella per ogni attributo determinante.
11. DROP cancella la tabella, TRUNCATE solo le righe, ALTER la modifica.
12. Le funzioni di aggregazione riducono tanti valori a uno; con GROUP BY se ne ottiene uno per gruppo.

6.2 Domande di autoverifica

Prova a rispondere senza guardare gli appunti, poi controlla nella pagina successiva.

1. Che differenza c'è tra uno schema ER e uno schema logico?
2. Perché le generalizzazioni vanno eliminate prima di passare allo schema logico?
3. In un'associazione (1,1)–(1,N), in quale tabella metti la FK e perché?
4. Che cosa puoi fare con un'associazione (0,1)–(1,N)?
5. Come si traduce l'associazione ricorsiva «un ministro succede a un altro»?
6. Impiegato lavora in più città e risiede in una: quante tabelle servono e dove vanno le FK?
7. Perché la matricola da sola non basta a identificare uno studente?
8. Che cosa impedisce il vincolo di integrità referenziale? Fai un esempio.

9. Nella tabella ANAGRAFICA(id, nome, cap, città) quale forma normale non è rispettata e come la correggi?
10. In che cosa differiscono una dipendenza parziale e una dipendenza transitiva?
11. Che differenza c'è tra DROP TABLE e TRUNCATE TABLE?
12. Perché la UNION richiede lo stesso numero e tipo di colonne, mentre la JOIN no?

6.3 Risposte

1. L'ER è un disegno concettuale della realtà; lo schema logico elenca tabelle, attributi e chiavi ed è pronto per essere implementato in un DBMS.
2. Perché le tabelle non hanno il concetto di gerarchia padre–figlio: la generalizzazione va trasformata in tabelle normali, senza perdere informazioni.
3. Nella tabella del lato (1,1): ogni sua riga è collegata a una sola riga dell'altra, quindi basta una colonna.
4. Mettere la FK nel lato (0,1) rendendola nullable, oppure creare una tabella dedicata all'associazione.
5. Con la tabella SUCCESSIONE(ministro_prec, ministro_succ): due FK che puntano entrambe a MINISTRO.
6. Tre: IMPIEGATO con la FK citta_residenza, CITTA, e LAVORA con le FK impiegato e citta (perché è molti a molti).
7. Perché la matricola è unica solo dentro un'università: serve anche l'università, che entra nella PK come FK.
8. Impedisce che una FK contenga un valore senza corrispondenza: per esempio citta_residenza = 57 quando non esiste la città con id 57.
9. La 3NF: città dipende da cap, che non è chiave (dipendenza transitiva). Si decompone in ANAGRAFICA(id, nome, cap) e CAP(cap, città).
10. La parziale è da una parte di una PK composta (riguarda la 2NF); la transitiva è da un attributo non chiave (riguarda la 3NF).
11. DROP elimina la tabella con i dati; TRUNCATE elimina solo le righe e la tabella resta, vuota.
12. Perché la UNION mette le righe una sotto l'altra, quindi le colonne devono combaciare; la JOIN affianca colonne di tabelle diverse.

Cosa è stato corretto rispetto agli appunti

Argomento	Negli appunti	Versione corretta
COUNT	«COUNT conta i valori non null»	Solo COUNT(campo) ignora i NULL; COUNT(*) conta tutte le righe, anche quelle con NULL.
Dipendenza funzionale	CAP → città: «questo è da evitare»	La dipendenza funzionale in sé non è un errore; va evitata quando il determinante (qui il CAP) non è chiave, perché è una dipendenza transitiva.